App.tsx Logic — The Stage Manager of InventoryPlus

A detailed explanation of how view rendering, sidebar navigation state, and modular UI sets are dynamically orchestrated in App.tsx.

1. Understanding the Core Layout & Routing

In a single-page React application, instead of loading a new HTML page from a server when a link is clicked, the main app component monitors the selection state and dynamically swaps the visible component.

What is happening here step-by-step:

- **The Sidebar Navigation:** The <SidebarNavigationSlimDemo /> is rendered on the left. It is passed the current selection states (activeIndex and activeSub) and state modifiers (handleItemSelect and handleSubSelect). When the user clicks on the sidebar, these callback functions run, updating the state variables and causing a component re-render.
- **The Main Content Frame:** The <main> element is the container on the right. Its class list dynamically updates if the showRightSidebar drawer is open (adding 'sidebar-open' to adjust layout widths).
- **Ternary Router:** The expression {isInventoryView ? <Inventory /> : <Placeholder />} decides what set is wheeled out onto the screen. If isInventoryView evaluates to true, the full <Inventory /> component is loaded. Otherwise, the screen displays a placeholder layout.

2. Full Source Code of App.tsx

The following is the complete, current source code of src/App.tsx:

```
import { useState } from 'react';
import { SidebarNavigationSlimDemo } from './features/navigation/Sidebar';
import { navItemsDualTier } from './features/navigation/constants';
import Inventory from './components/Inventory';
import Dashboard from './components/Dashboard';
import { useLocalStorage } from './shared/hooks/useLocalStorage';
import './App.css';

function App() {
  const [activeIndex, setActiveIndex] = useLocalStorage("inventoryplus_active_index", 0);
  const [activeSub, setActiveSub] = useLocalStorage("inventoryplus_active_sub", "Overview");
  const [showRightSidebar, setShowRightSidebar] = useLocalStorage("inventoryplus_show_right_sidebar", false);
  const [resetKey, setResetKey] = useState(0);

  const handleItemSelect = (index: number) => {
    setActiveIndex(index);
    // Sync sub-item to the first sub-item of the selected category
    const item = navItemsDualTier[index];
    if (item && item.items && item.items.length > 0) {
      setActiveSub(item.items[0].label);
    } else {
      setActiveSub("");
    }
    // Clear selection if explicitly clicking sidebar tab
    localStorage.removeItem("inventoryplus_selected_department");
    localStorage.removeItem("inventoryplus_selected_member_index");
    localStorage.removeItem("inventoryplus_selected_member_username");
    localStorage.removeItem("inventoryplus_show_right_sidebar");
    setShowRightSidebar(false);
    setResetKey(prev => prev + 1);
  };

  const handleSubSelect = (label: string) => {
    setActiveSub(label);
    // Clear selection if explicitly clicking sidebar sub-select
    localStorage.removeItem("inventoryplus_selected_department");
    localStorage.removeItem("inventoryplus_selected_member_index");
    localStorage.removeItem("inventoryplus_selected_member_username");
    localStorage.removeItem("inventoryplus_show_right_sidebar");
    if (label === "Overview" || label === "Products" || label === "Inventory") {
      setShowRightSidebar(false);
      setResetKey(prev => prev + 1);
    }
  };

  const isInventoryView = activeIndex === 1 || (activeIndex === 0 && (activeSub === "Overview" || activeSub === "
Products" || activeSub === "Inventory"));
  const isDashboardView = activeIndex === 2;

  const getCurrentViewName = () => {
    const mainItem = navItemsDualTier[activeIndex];
    if (!mainItem) return "Section";
    if (mainItem.items && mainItem.items.length > 0 && activeSub) {
      const hasSub = mainItem.items.some(sub => sub.label === activeSub);
      if (hasSub) {
        return `${mainItem.label} — ${activeSub}`;
      }
    }
    return mainItem.label;
  };
```

```
  return (
    <div className="app-layout">
      <SidebarNavigationSlimDemo
        activeIndex={activeIndex}
        onItemSelect={handleItemSelect}
        activeSub={activeSub}
        onSubSelect={handleSubSelect}
      />
      <main className={`app-main ${showRightSidebar ? 'sidebar-open' : ''}`} style={{ display: 'flex', flexDirect
ion: 'column', alignItems: 'flex-start', gap: '20px', width: '100%' }}>
        {isInventoryView ? (
          <Inventory
            key={resetKey}
            showRightSidebar={showRightSidebar}
            setShowRightSidebar={setShowRightSidebar}
          />
        ) : isDashboardView ? (
          <Dashboard />
        ) : (
          <div style={{ padding: '40px', textAlign: 'center', width: '100%', color: 'var(--text)' }}>
            <h2 style={{ fontSize: '24px', fontWeight: 600, color: 'var(--text-h)' }}>
              {getCurrentViewName()} View
            </h2>
            <p style={{ marginTop: '10px', opacity: 0.7 }}>
              This section is currently under development. Click on <b>Overview</b> or the <b>Inventory</b> icon
in the sidebar to see the Profile Cards.
            </p>
          </div>
        )}
      </main>
    </div>
  );
}

export default App;
```

3. The Stage-Manager Analogy

To easily conceptualize how this works, think of your application layout as a **theater stage production**:

React Concept	Theater Analogy	Role / Description
App.tsx	**The Stage Manager**	Stands backstage holding the script, coordinating which scene should be presented next.
Sidebar	**The Lobby Directory**	The board listing all the plays and showtimes. The audience points to an item on this board to select a show.
activeIndex / activeSub	**The Selected Scene ID**	The specific play or scene number the audience has chosen to view.
<Inventory />	**The Detailed Castle Set**	A complex, pre-constructed stage set with props (tables, department members, spec cards) wheeled out when called.
	The 'Under Construction' Curtain	A plain curtain with a simple sign pointing back to the completed rooms.

How they work together:
When a user clicks on the **Lobby Directory** (Sidebar), the audience tells the **Stage Manager** (App.tsx): *'We want to see Scene 1 (Inventory)!'*. The Stage Manager immediately updates the **Scene ID** (state), clears away any previous props, and instructs the crew to wheel out the **Castle Set** (<Inventory />) onto the stage.

If the audience clicks on a scene that hasn't been built yet (like Scene 2 - Dashboard), the Stage Manager wheels out the **'Under Construction' Curtain** instead.

4. What Happens If We Add a Dashboard?

If you want to implement the Dashboard, you are simply adding a brand new stage set (a <Dashboard /> component) and instructing the Stage Manager to wheel it out when the corresponding tab is selected.

Step 1: Build the New Stage Set (Dashboard.tsx)

Create a standalone component file Dashboard.tsx containing the layout grid, cards, and charts. This isolates the dashboard code from the rest of the app.

Step 2: Tell the Stage Manager (App.tsx) About It

Import the new component at the top of App.tsx and update the condition checks. Here is a code diff showing exactly how the routing changes from a binary choice (Inventory vs. Placeholder) to a multi-view router:

```
// 1. Import the new component at the top of App.tsx
import Inventory from './shared/inventory/Inventory';
+ import Dashboard from './features/dashboard/Dashboard';
...
// 2. Define the routing check
const isInventoryView = activeIndex === 1 || ...;
+ const isDashboardView = activeIndex === 2; // Dashboard is index 2 in constants.ts
...
// 3. Update the render tree in the return block
<main className="app-main">
{isInventoryView ? (
<Inventory key={resetKey} ... />
+ ) : isDashboardView ? (
+ <Dashboard />
) : (
<div className="placeholder-view">
<h2>{getCurrentViewName()} View</h2>
<p>This section is currently under development...</p>
</div>
)}
</main>
```

By nesting the conditional (or replacing it with a switch statement as the app grows), you give the Stage Manager clear rules on exactly which component matches each user click.